

Estrategia del Jugador Autónomo para HEX

Adriana Boué García

Marzo 2026

Este documento describe la estrategia y los detalles de implementación del agente **SmartPlayer** para el juego HEX. El diseño del agente se centra en maximizar la eficiencia computacional bajo restricciones de tiempo, utilizando búsqueda adversaria y una heurística basada en la teoría de caminos mínimos.

Estrategia de Juego y Apertura

Control del Centro

El agente implementa una jugada de apertura optimizada: si el tablero se encuentra vacío, se selecciona automáticamente la coordenada central ($\lfloor size/2 \rfloor, \lfloor size/2 \rfloor$). Esta decisión se fundamenta en que el centro posee el mayor grado de libertad en un tablero hexagonal, facilitando la conexión hacia cualquier extremo y obligando al oponente a reaccionar de forma defensiva desde el primer turno.

Generación y Filtrado de Movimientos

Dado que el factor tiempo es crítico (límite de 5 segundos por jugada), **SmartPlayer** emplea una técnica de reducción del espacio de búsqueda mediante:

- **Heurística de Proximidad (moves):** Solo se consideran celdas vacías que tengan al menos un vecino ocupado. Esto evita explorar zonas irrelevantes del tablero y concentra el esfuerzo en los frentes de expansión.
- **Táctica de Puentes (bridge moves):** Se priorizan los patrones de conexión indirecta, donde dos piezas propias están separadas por un espacio que permite una conexión virtual. Esta formación es fundamental en HEX, ya que garantiza un enlace que el oponente no puede bloquear fácilmente con una sola jugada.

Algoritmo de Decisión

El Método play: El Orquestador

El método **play** actúa como el motor principal de la toma de decisiones. Su flujo lógico se divide en tres fases críticas:

1. **Pre-selección:** Combina los movimientos de proximidad y los puentes para generar una lista de candidatos.
2. **Ordenamiento Heurístico:** Antes de invocar la búsqueda profunda, cada movimiento candidato es evaluado de forma superficial mediante `evaluate_move`. Esto permite ordenar las jugadas de mejor a peor, lo que maximiza la probabilidad de que la poda alfa-beta descarte ramas inútiles rápidamente.
3. **Ejecución:** Se seleccionan las 15 mejores jugadas y se someten al análisis del algoritmo Minimax.

Búsqueda Minimax con Poda Alfa-Beta

La lógica adversaria se implementa mediante un algoritmo Minimax que simula el escenario de que ambos jugadores juegan de forma óptima.

- **Poda Alfa-Beta:** Se utilizan los parámetros α (mejor opción para el maximizador) y β (mejor opción para el minimizador) para interrumpir la evaluación de nodos que no pueden mejorar el resultado ya descubierto.
- **Recursividad Controlada:** El algoritmo desciende hasta una profundidad de 2 niveles, permitiendo al agente prever la respuesta del oponente y su propia contra-respuesta.

Función de Evaluación Heurística

La calidad de un estado se define mediante la métrica de "distancia de conexión". La función de evaluación es:

$$f(s) = \text{Distancia}_{rival} - \text{Distancia}_{propia} \quad (1)$$

Cálculo de Distancia con 0-1 BFS (aux)

Para que la evaluación sea precisa y rápida, se utiliza el método `aux` con una `deque` (cola de doble extremo) para implementar un algoritmo de camino mínimo eficiente:

- **Costo 0 (`appendleft`):** Si una celda vecina ya está ocupada por el jugador, la distancia no aumenta, otorgándole prioridad máxima en la cola.
- **Costo 1 (`append`):** Si la celda está vacía, representa un movimiento necesario para conectar, sumando 1 a la distancia total.
- **Bloqueo:** Las piezas del oponente se consideran obstáculos insuperables, obligando al algoritmo a buscar rutas alternativas.

Conclusión

`SmartPlayer` combina la agresividad táctica del control central con la robustez algorítmica del Minimax y 0-1 BFS. Esta arquitectura garantiza que cada movimiento no solo busque la conexión propia, sino que simultáneamente incremente la dificultad de conexión para el rival, todo dentro de los márgenes de tiempo establecidos.